Lecture 12 Questions: Virtual Calls, DFS vs. BFS

CSCI 235

Question 3: why does `reachable()` run the right search?

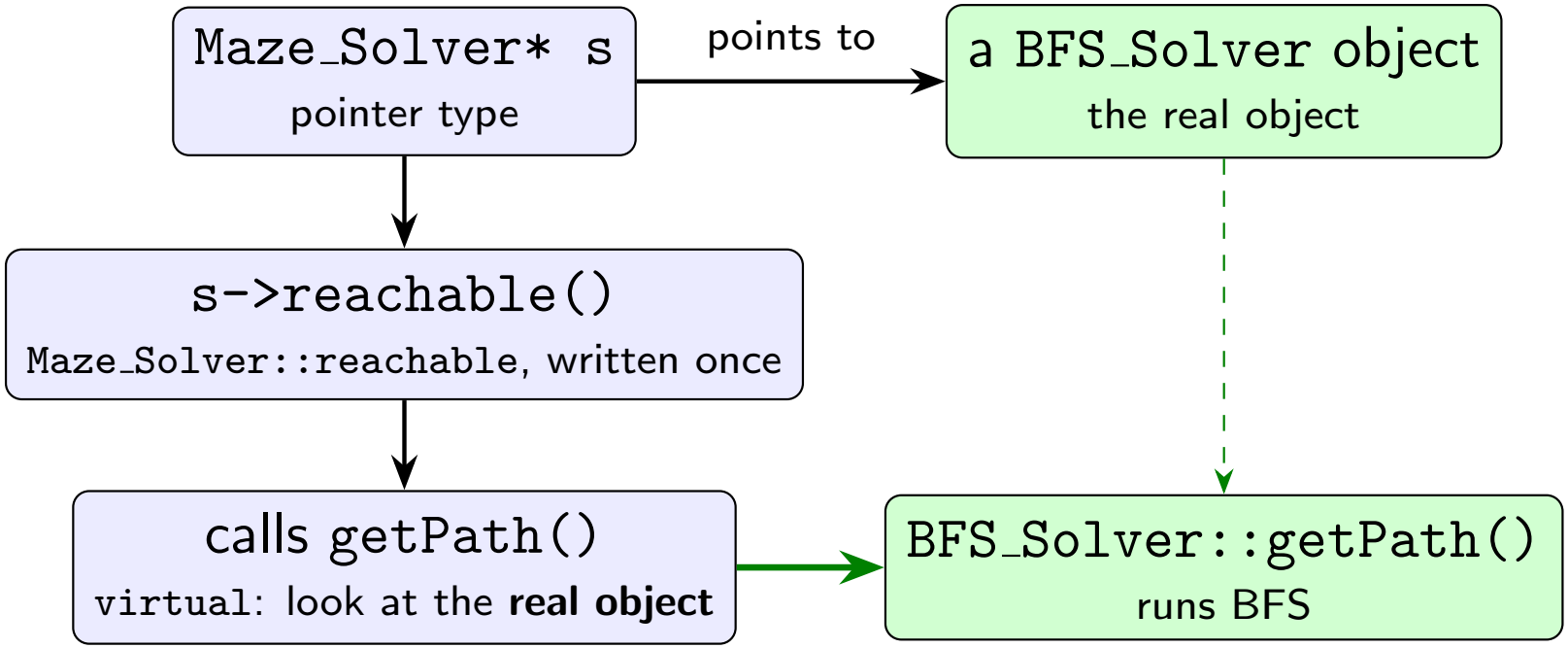
```
class Maze_Solver {
public:
    bool reachable() const;
    virtual std::stack<Coord> getPath() const = 0;
    virtual ~Maze_Solver() = default;
};

bool Maze_Solver::reachable() const {
    return !getPath().empty();
}
```

The puzzle

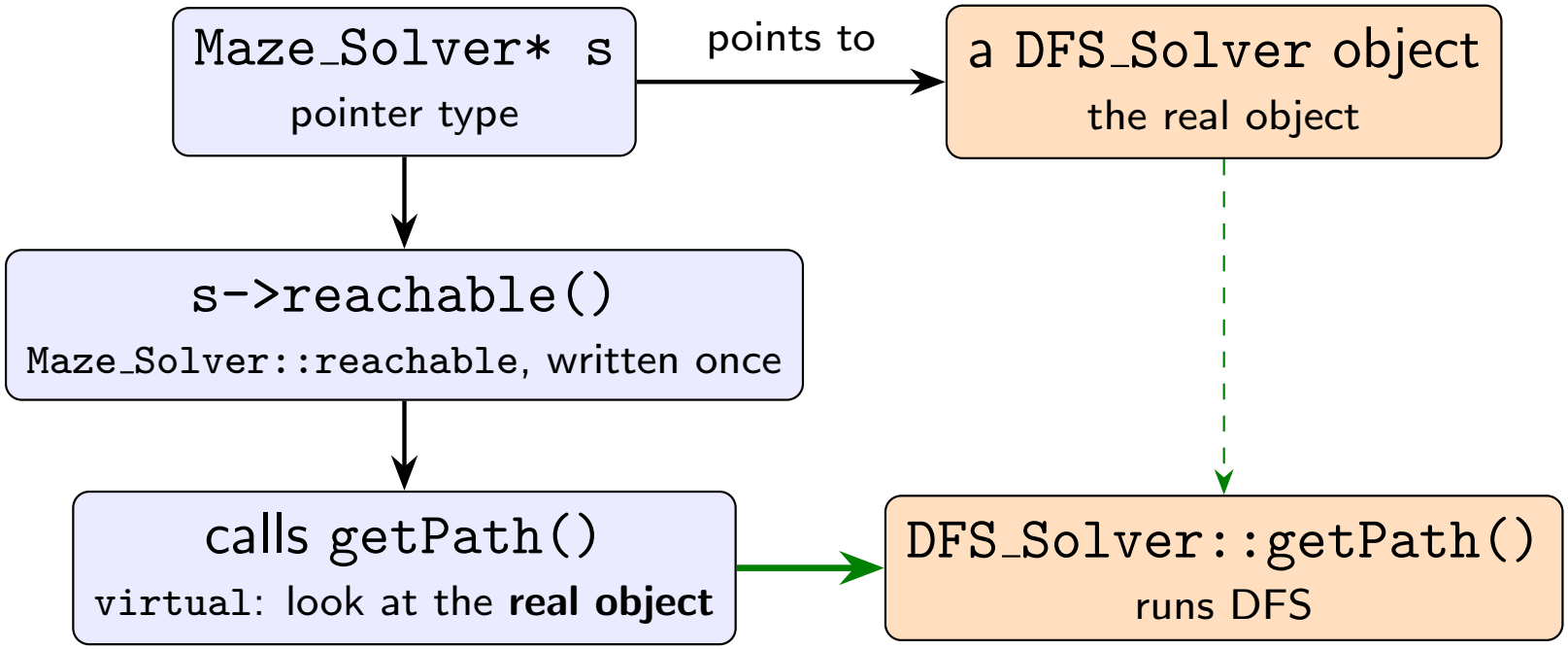
`reachable()` is written **once**, in `Maze_Solver`. Yet for a `BFS_Solver` it runs BFS, and for a `DFS_Solver` it runs DFS. How?

The call chain when `s` points to a `BFS_Solver`



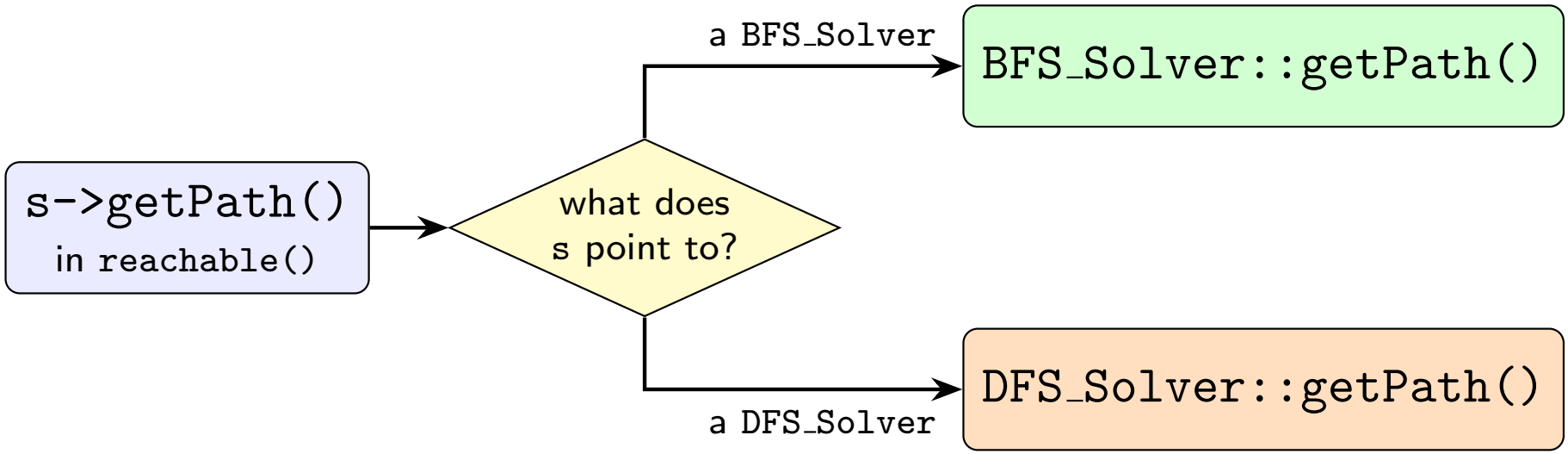
The **pointer type** says which functions you may call. The **real object** decides which version of a virtual function runs, and it is looked up **at run time**.

Same code when `s` points to a `DFS_Solver`



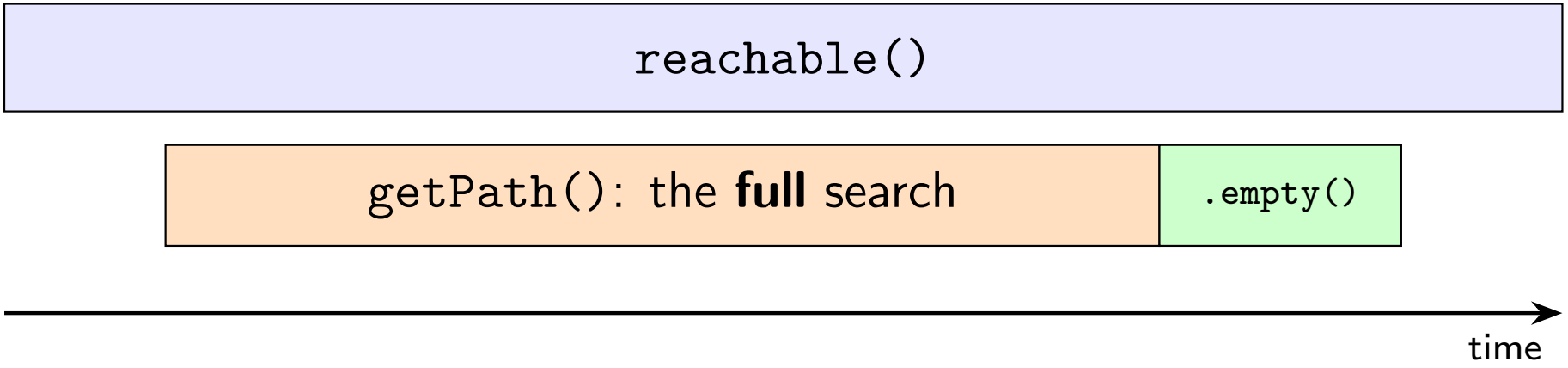
Nothing in `reachable()` changed. Only the object behind `s` changed, so the virtual call now lands in `DFS_Solver::getPath()`.

A virtual call acts like a switch on the real object



One line of code, **different behavior** depending on the real object: this is **polymorphism**. Think of it as a switch, with one difference: you do not write the `if`. C++ looks the answer up for you at run time, and a new solver class just adds a new branch.

`reachable()` does not just peek: it runs the whole search



A common misconception

"`reachable()` tells us if a path exists, but doesn't technically find it." **Not quite:** it **does** find the path. It calls `getPath()`, which runs the entire search, and then only asks whether the result is empty.

Consequence

Calling `reachable()` and then `getPath()` runs the search **twice**. If you need the route, call `getPath()` once and test `.empty()` yourself.

`virtual`, `= 0`, **and** `override`

`virtual`	Run-time choice: C++ looks at the **real object** to pick the function. This is what gives **dynamic binding**.
`= 0` **(pure)**	The function is **pure virtual**. The class becomes **abstract**, so you cannot create a plain `Maze_Solver`. A concrete derived solver **must** provide its own `getPath()`.
`override`	In a derived class: "I am replacing a virtual function." The compiler checks the name and signature match.
`virtual ~Maze_Solver()`	So `delete s` through a `Maze_Solver*` destroys the **whole** derived object.

Two students had it right

"`getPath()` is a pure virtual function, which helps with dynamic binding."
Precisely: `virtual` gives the run-time lookup; `= 0` makes the class abstract.

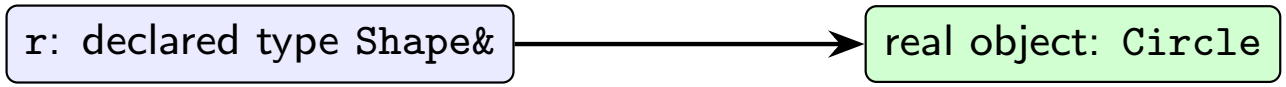
Question 2(b): what if we remove `virtual`?

```
struct Shape {
  virtual std::string name() const
    { return "Shape"; }
};
struct Circle : Shape {
  std::string name() const override
    { return "Circle"; }
};
Circle c;
Shape& r = c;
std::cout << r.name();
```

With `virtual`: prints `Circle`.

Remove `virtual` (and `override`): prints `Shape`.

Why: without `virtual`, the compiler decides from the **declared type** of `r`, which is `Shape&`. It never looks at the `Circle` underneath.



no `virtual`: decided at **compile time** from the declared type ⇒ `Shape::name`

`virtual`: decided at **run time** from the real object ⇒ `Circle::name`

Note: why not `struct Circle : public Shape`?

A `struct` is **public by default**, for its members and for inheritance, so `struct Circle : Shape` already means `: public Shape`. With `class`, the default is **private**, so you must write `class Circle : public Shape`. (Same pattern as Q2(b) on the slip; your names may differ.)

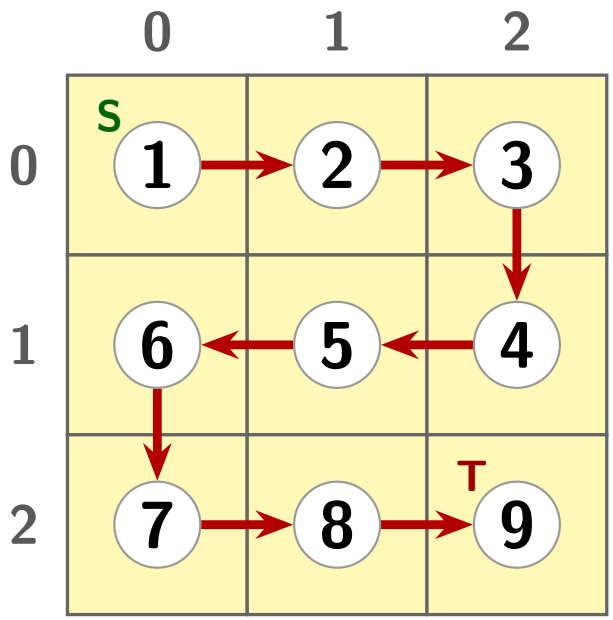
Question: what is the difference between DFS and BFS?

	DFS	BFS
Idea	go as deep as possible, then back up	spread out in layers by distance
Container	call stack (recursion)	queue
Next cell explored	continues from the newest (deepest) call	the oldest waiting cell
Route returned	**a** route, maybe long	a **shortest** route
Time	$O(n)$	$O(n)$
Space	$O(n)$, deep recursion	$O(n)$, parent table + queue

Analogy

DFS is a hiker who follows one trail until it ends. BFS is a ripple

DFS on an empty 3×3 grid

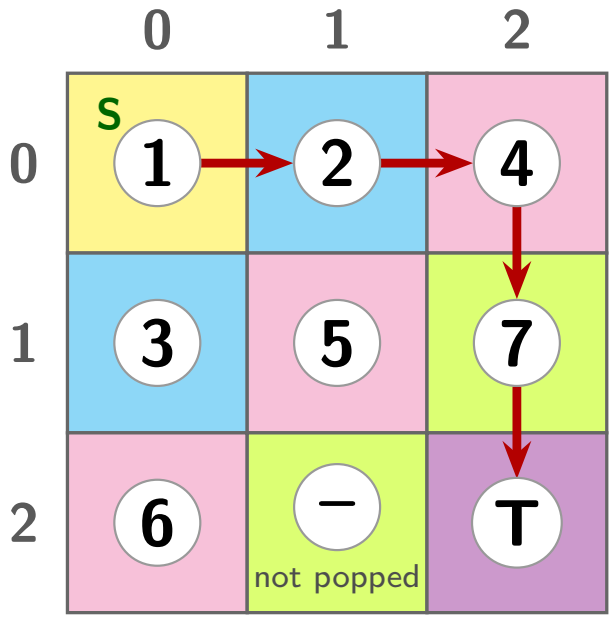


Neighbor order: left, right, up, down. Numbers show the order cells are **entered**.

From $(0, 0)$: right, right, then down to $(1, 2)$. At $(1, 2)$ DFS tries **left** before down, so it sweeps back through the middle and bottom rows and finally reaches T.

Every active call succeeds, so **all 9 cells** are on the returned route.

BFS on the same grid

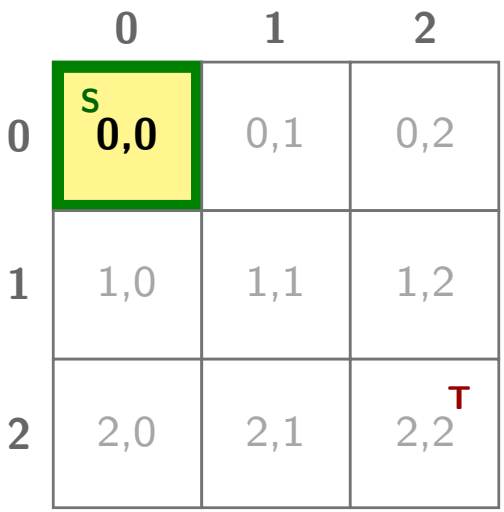


Numbers show the order cells are **popped**. Colors show the distance layer (same color = same distance from S).

BFS stops the moment T is **discovered**: it is found while processing $(1, 2)$, and T is never pushed.

The red arrows follow the parent table back from T: **5 cells**.

BFS, step 1 of 9



legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back



BFS search tree (parent relationships)

layer 0

layer 1

layer 2

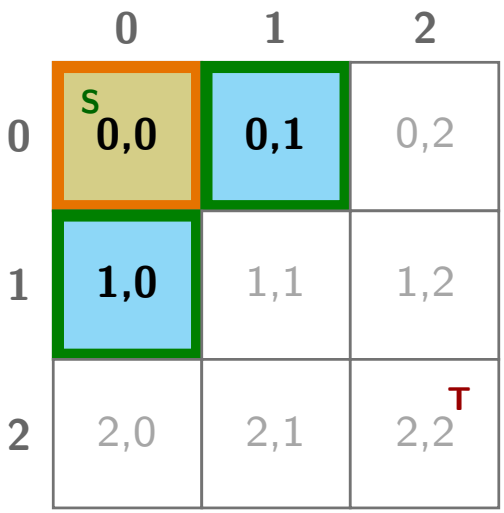
layer 3

layer 4



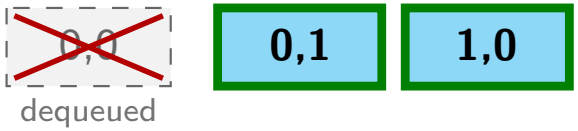
Step 1. Enqueue the start $(0, 0)$, layer 0. Mark it visited. Queue: $(0, 0)$.

BFS, step 2 of 9



legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back



dequeued

BFS search tree (parent relationships)

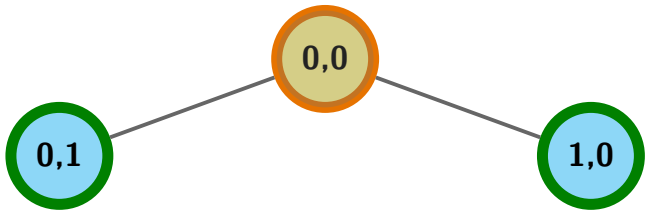
layer 0

layer 1

layer 2

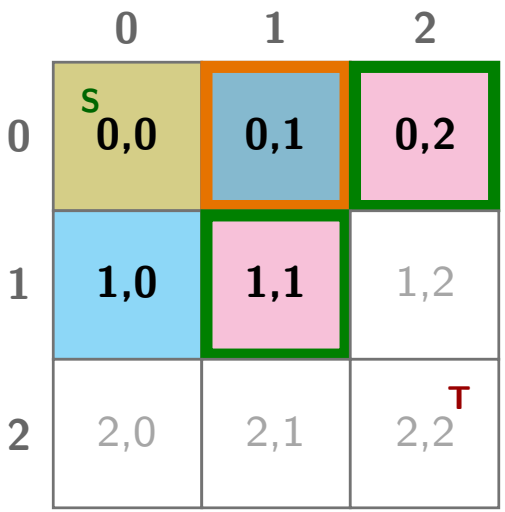
layer 3

layer 4



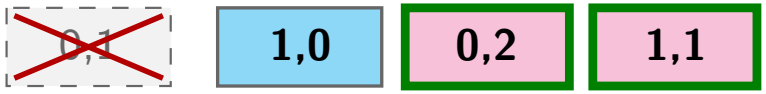
Step 2. Dequeue $(0, 0)$ and gray it. Look left, right, up, down: right $(0, 1)$ and down $(1, 0)$ are open and unvisited, so enqueue them (layer 1). $(0, 0)$ is their **parent**.

BFS, step 3 of 9



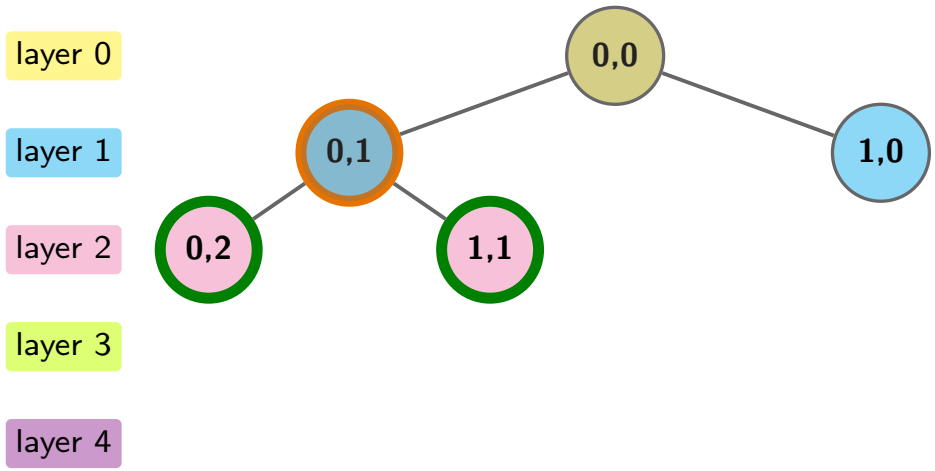
legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back



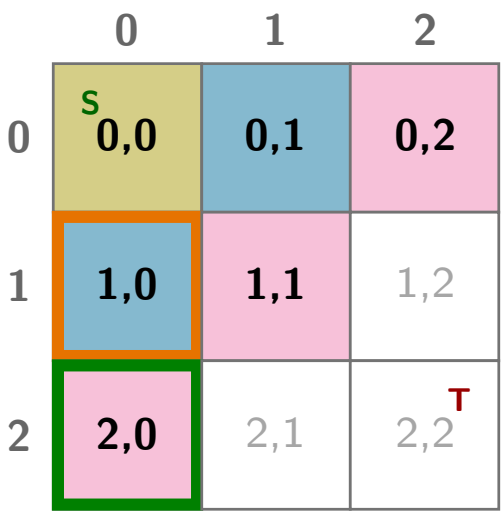
dequeued

BFS search tree (parent relationships)



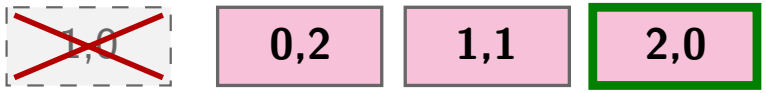
Step 3. Dequeue $(0, 1)$. Add right $(0, 2)$ and down $(1, 1)$ (layer 2). Parent of both: $(0, 1)$.

BFS, step 4 of 9



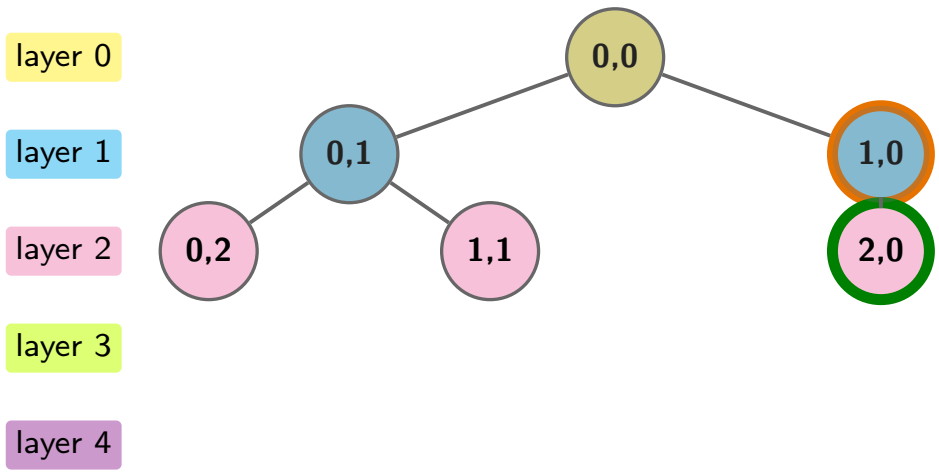
legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back



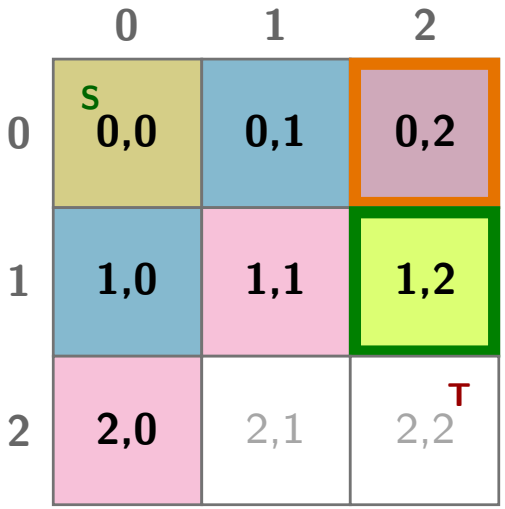
dequeued

BFS search tree (parent relationships)



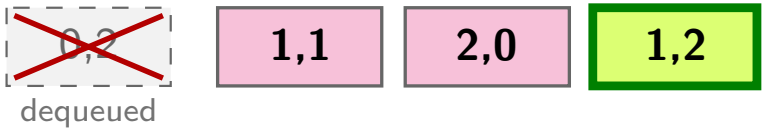
Step 4. Dequeue $(1, 0)$. Add down $(2, 0)$. Its right neighbor $(1, 1)$ is **already in the queue**: every cell is marked visited when it is added, so it is not added again.

BFS, step 5 of 9

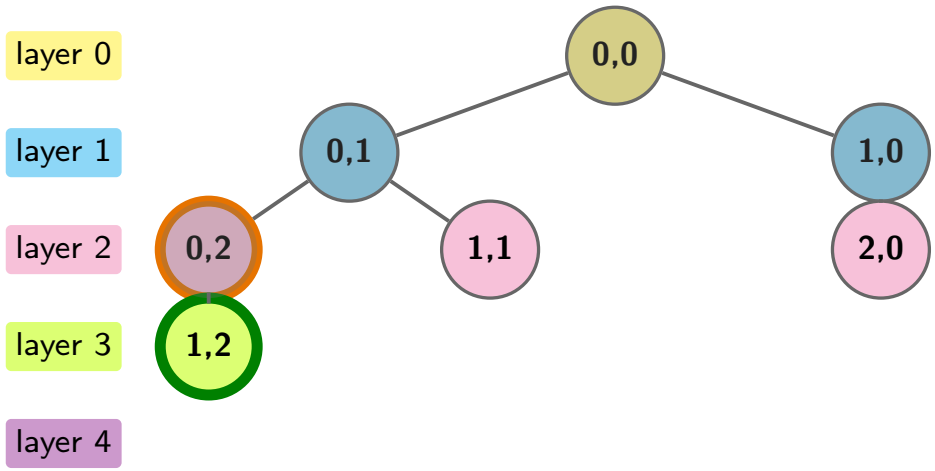


legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back

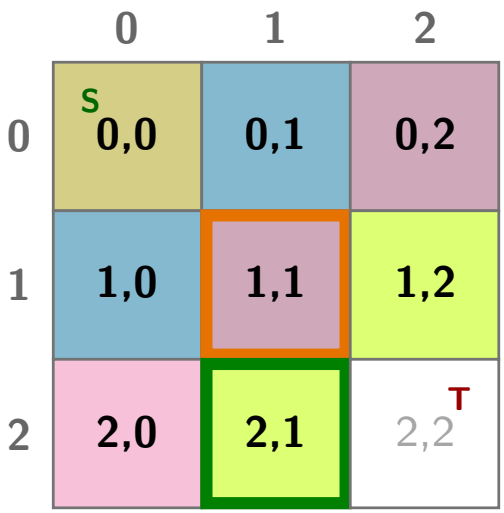


BFS search tree (parent relationships)



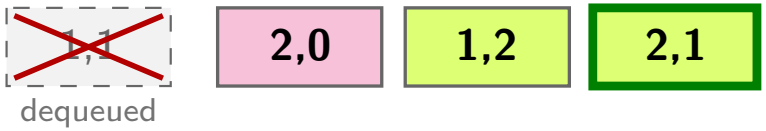
Step 5. Dequeue $(0, 2)$. Add down $(1, 2)$ (layer 3).

BFS, step 6 of 9

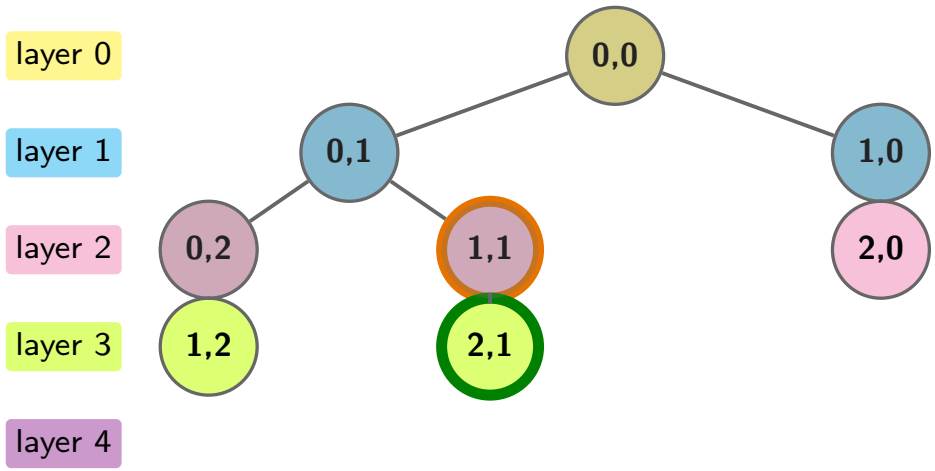


legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back

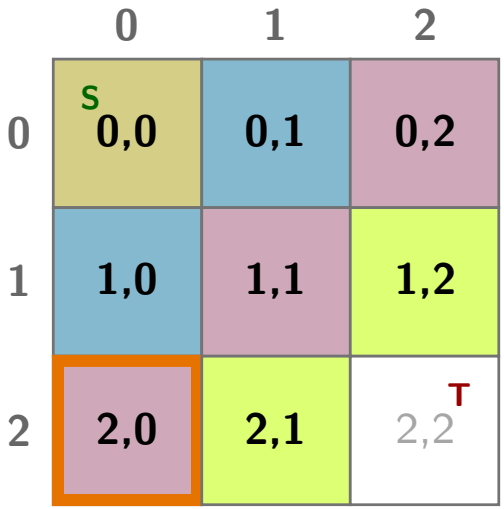


BFS search tree (parent relationships)



Step 6. Dequeue $(1, 1)$. Add down $(2, 1)$ (layer 3). Left, right and up are visited.

BFS, step 7 of 9



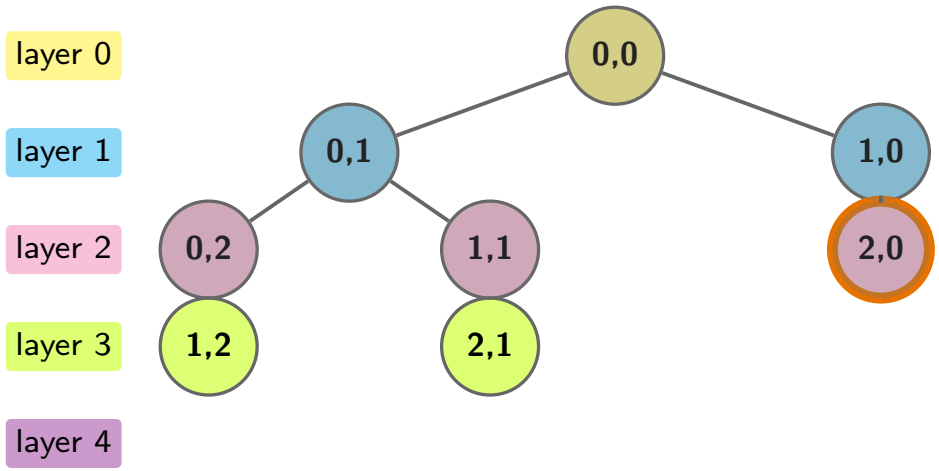
legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back



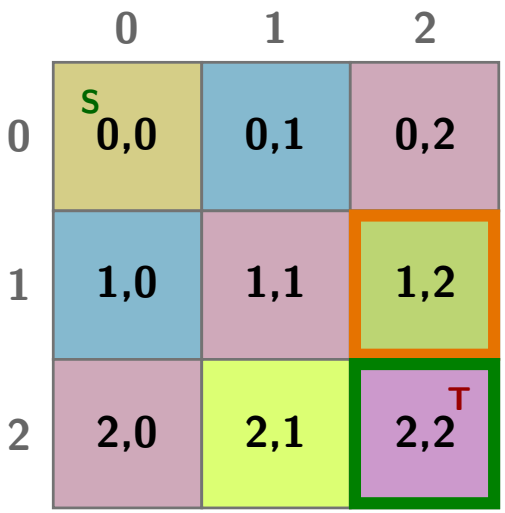
dequeued

BFS search tree (parent relationships)



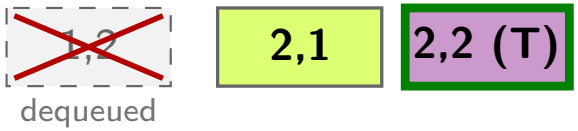
Step 7. Dequeue $(2, 0)$. No unvisited open neighbor, so nothing is added.

BFS, step 8 of 9

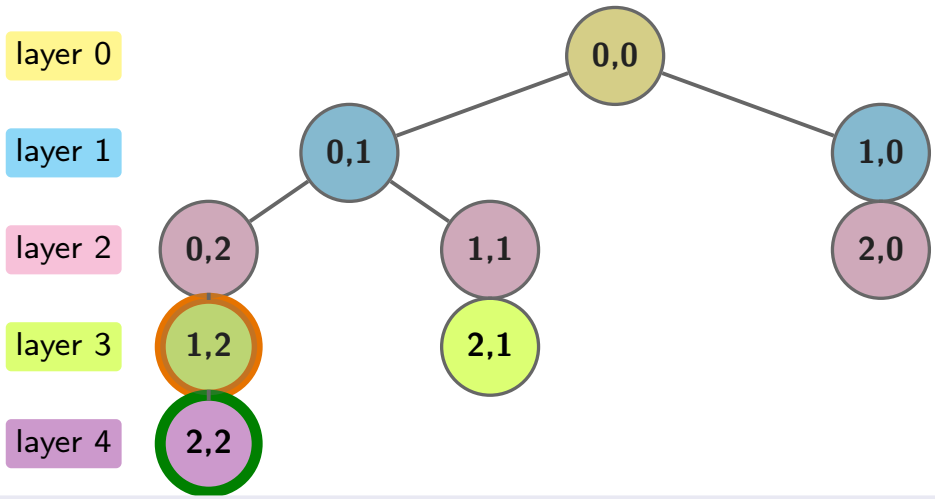


legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back

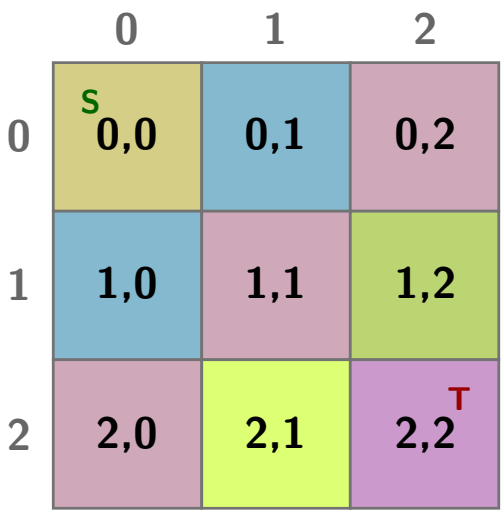


BFS search tree (parent relationships)



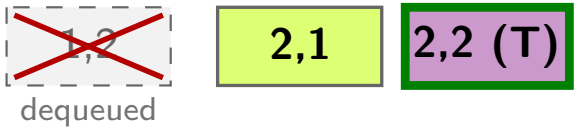
Step 8. Dequeue $(1, 2)$. Its down neighbor $(2, 2)$ is the **target**: found! (Our code stops here, so T need not even be pushed, and $(2, 1)$ is never dequeued.)

BFS, step 9 of 9: rebuild the path

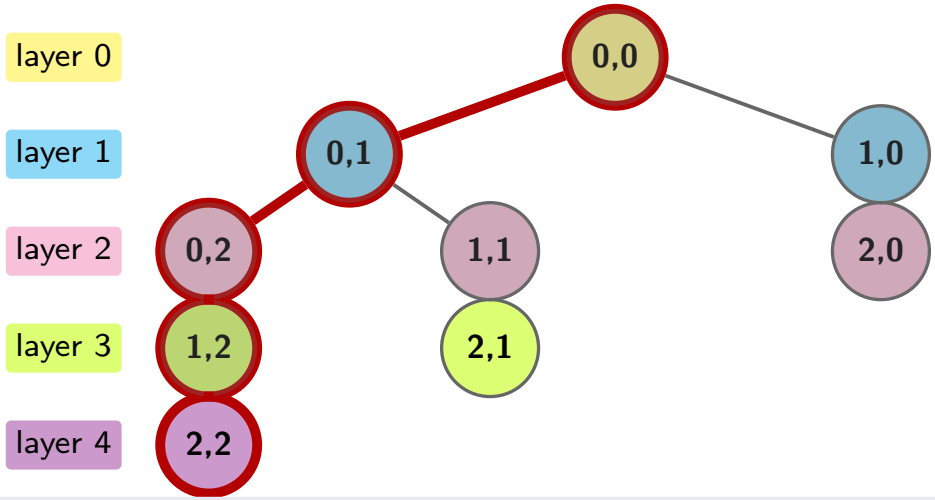


legend: **orange** = just dequeued, **green** = just added, processed cells are darkened, colors = layers (see the layer labels).

Queue front → back



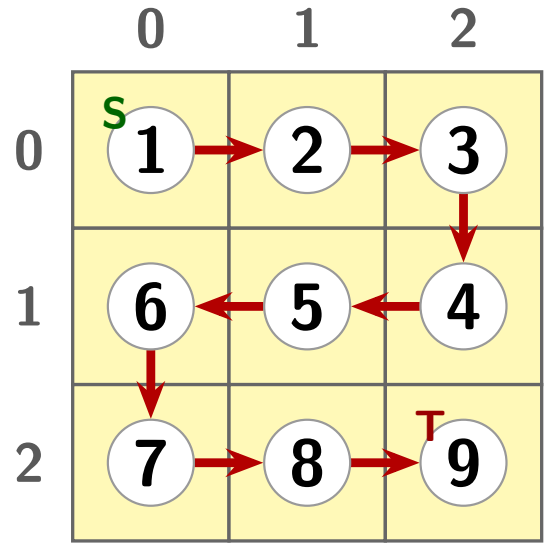
BFS search tree (parent relationships)



Step 9. Follow parents from T:
$(2,2) \leftarrow (1,2) \leftarrow (0,2) \leftarrow (0,1) \leftarrow (0,0)$. Push each onto `pathStack` (end first), so the start is on top: **5 cells**, a shortest route.

Side by side: same maze, same start, same target

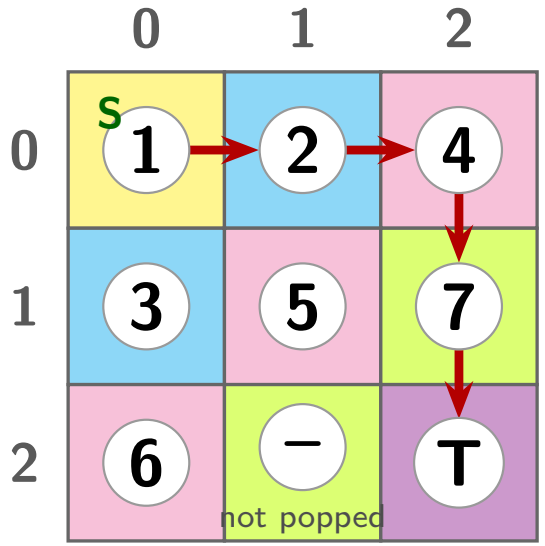
DFS



route: **9 cells**

a route, not the shortest

BFS



route: **5 cells**

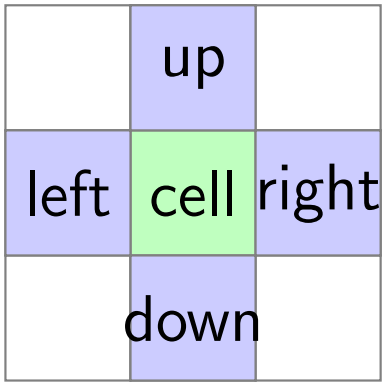
a shortest route

What the queue looks like during BFS

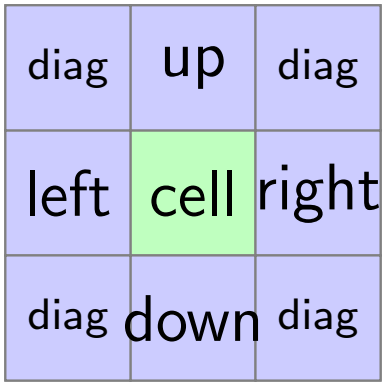
step	queue after the step (front on the left)		
pop $(0,0)$	(0,1)	(1,0)	
pop $(0,1)$	(1,0)	(0,2)	(1,1)
pop $(1,0)$	(0,2)	(1,1)	(2,0)
pop $(0,2)$	(1,1)	(2,0)	(1,2)
pop $(1,1)$	(2,0)	(1,2)	(2,1)
pop $(2,0)$	(1,2)	(2,1)	
pop $(1,2)$	T discovered: stop		

The queue always holds cells of distance k followed by cells of distance $k+1$: the ripple. DFS would instead take the **newest** cell each time, so it would dive.

Question: why not check diagonal neighbors?



Project 2: **4** neighbors

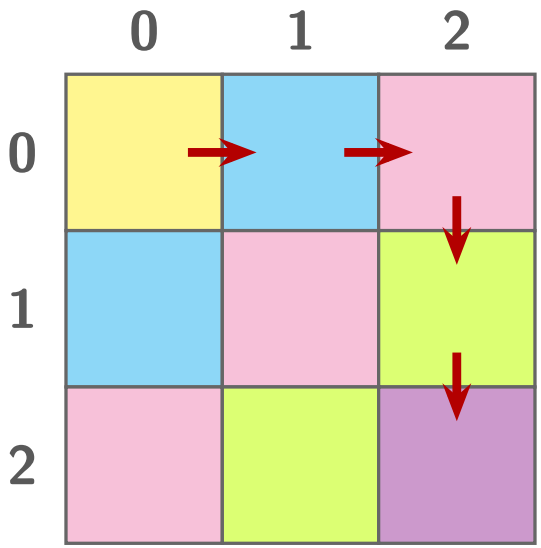


A variant: **8** neighbors

This is **not a limit of BFS (or DFS)**. Project 2 **defines** a move as left, right, up or down, so the code lists four neighbors. Nothing stops you from listing eight, but then the problem is different: it changes which route counts as "shortest."

Same grid, two different rules

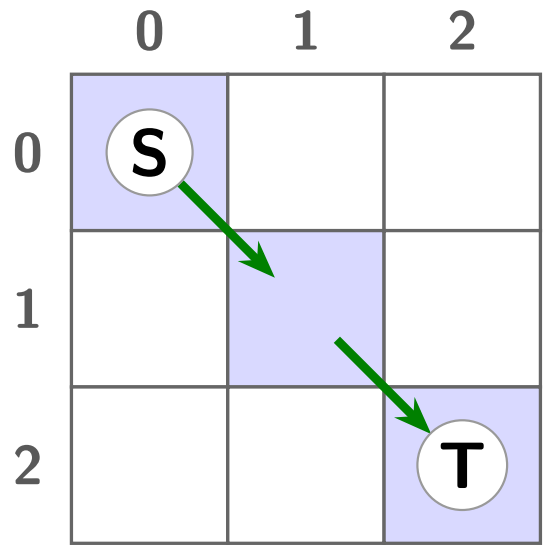
4 neighbors



BFS route: **5 cells**

4 moves

8 neighbors



BFS route: **3 cells**

2 moves

To try it, the neighbor list in BFS would use eight offsets: `{0,-1},{0,1},{-1,0},{1,0},{-1,-1},{-1,1},{1,-1},{1,1}`. Design question: may a diagonal move squeeze between two walls? Project 2 avoids the question by allowing only four moves.

Summary and practice

- `reachable()` calls the **virtual** `getPath()`, so C++ runs the version of the **real object** (dynamic binding).
- `reachable()` runs the **whole** search. It is not a quick peek.
- Without `virtual`, the **declared type** decides, at compile time.
- DFS: call stack (recursion), **a** route. BFS: queue, a **shortest** route. Both $O(n)$ time.
- Diagonals are a **rule choice**, not a BFS limitation.

Practice (more practice was requested)

`class12_flashcards.pdf` and `bfs_card_quest.html` on the course site (`supplements/`).